Classification Methods

Md. Aminul Islam Shazid

Outline

1. Classification Problems and Decision Boundaries
2. Logistic Regression
3. Likelihood Function
4. Classification with More than Two Classes
5. k-Nearest Neighbors
6. Comparison and Summary

Classification Problems and Decision Boundaries

The Classification Problem

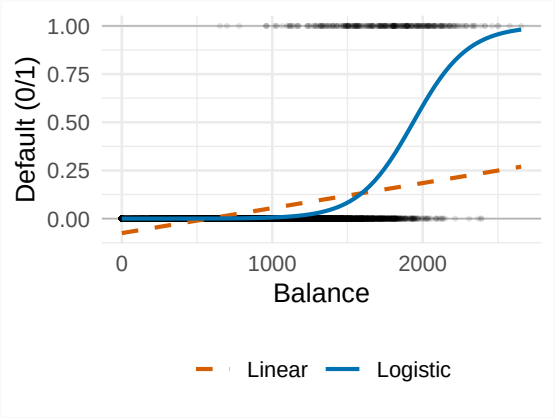
- The response $y^{(i)}$ is qualitative: it takes one of K classes, $y^{(i)} \in \{1, \dots, K\}$, or $y^{(i)} \in \{0, 1\}$ when $K = 2$.
- Training data: $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$ with feature vectors $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_p^{(i)})^\top \in \mathbb{R}^p$.
- Goal: learn a classifier $f : \mathbb{R}^p \to \{1, \dots, K\}$ such that $\hat{y}^* = f(\mathbf{x}^*)$ is correct for a new observation $\mathbf{x}^*$.
- Intercept notation: $\tilde{\mathbf{x}}^{(i)} = (1, \mathbf{x}^{(i)\top})^\top \in \mathbb{R}^{p+1}$ prepends a 1, and the design matrix $\tilde{\mathbf{X}} \in \mathbb{R}^{n \times (p+1)}$ has rows $\tilde{\mathbf{x}}^{(i)\top}$. Coefficient vectors such as $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^\top$ include the intercept.

Classification vs. Regression

- Regression predicts a quantitative response; classification predicts a category.
- Examples: spam vs. not spam, loan default, land-cover class from spectral bands, disease present vs. absent.
- Most classifiers first estimate a probability for each class, and then assign the observation to a class using these probabilities.

Why Not Linear Regression?

- Coding a binary response as $y^{(i)} \in \{0, 1\}$ and fitting $\hat{y} = \hat{\boldsymbol{\beta}}^\top \tilde{\mathbf{x}}$ gives fitted values that are not confined to $[0, 1]$, so they are not valid probabilities.
- For $K > 2$, coding the classes as $1, 2, 3, \ldots$ imposes an ordering and equal spacing that the classes do not have.
- We need a model whose output always lies in $[0, 1]$.



`Default` data: the linear fit is negative for small balances, the logistic fit stays in $[0, 1]$.

Measuring Classification Error

- A classifier's mistakes are counted by the error rate: the fraction of observations whose predicted class differs from the true class.
- With predictions $\hat{y}^{(i)}$ for $i = 1, \ldots, n$:

$$\text{error rate} = \frac{1}{n} \sum_{i=1}^{n} \mathbb{1}\{y^{(i)} \neq \hat{y}^{(i)}\}.$$

- The indicator $\mathbb{1}\{A\}$ equals 1 if the statement A is true and 0 otherwise, so the sum counts the mistakes. Example: 8 mistakes among 100 observations gives an error rate of 0.08.
- Computed on the training data this is the training error rate; what matters is the error on new observations, treated in a later lecture.

Conditional Class Probabilities

- Suppose we want to predict whether customers of a bank will repay their loans or will default on it.
- For a fixed $\mathbf{x}$ the class is still random: customers with identical balance, income and student status do not all default, or all repay.
- The conditional class probability $p_k(\mathbf{x}) = P(y = k \mid \mathbf{x})$ is the probability that the response is class k given the feature vector $\mathbf{x}$. Read it as the long-run proportion of observations with exactly this $\mathbf{x}$ that belong to class k.
- The probabilities of all classes sum to one: $\sum_{k=1}^{K} p_k(\mathbf{x}) = 1$.
- Example with $K = 3$ land-cover classes at a pixel with spectral values $\mathbf{x}$: $p_{\text{water}}(\mathbf{x}) = 0.1$, $p_{\text{forest}}(\mathbf{x}) = 0.6$, $p_{\text{urban}}(\mathbf{x}) = 0.3$.

The Bayes Classifier

- A natural rule: for each $\mathbf{x}$, predict the class with the highest conditional probability.

$$f_{\text{Bayes}}(\mathbf{x}) = \arg\max_{k \in \{1,\ldots,K\}} p_k(\mathbf{x}),$$

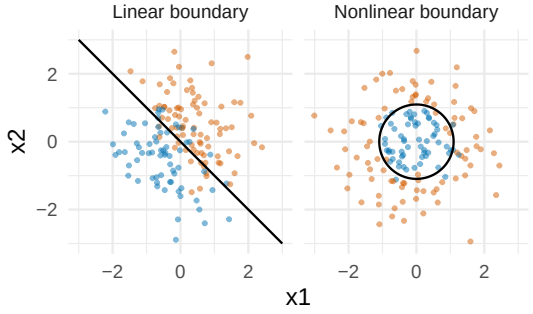
  where $\arg\max$ returns the class k at which $p_k(\mathbf{x})$ is largest.
- In the land-cover example the rule predicts forest, since 0.6 is the largest of the three probabilities.
- For two classes with $y \in \{0, 1\}$ and $p(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$: predict 1 if $p(\mathbf{x}) > 0.5$ and 0 otherwise.
- Among all classifiers, the Bayes classifier makes the fewest mistakes on average.

From the Bayes Classifier to Practical Classifiers

- The Bayes classifier is an unattainable gold standard: the true $p_k(\mathbf{x})$ is unknown.
- Practical classifiers estimate $\hat{p}_k(\mathbf{x})$ from the training data and plug it into the same rule: predict $\arg\max_k \hat{p}_k(\mathbf{x})$.
- Logistic regression estimates $p_k(\mathbf{x})$ with a parametric model whose parameters are fitted by maximum likelihood.
- k-nearest neighbors estimates $p_k(\mathbf{x})$ with class proportions among nearby training points.

Decision Boundaries

- A classifier splits the feature space $\mathbb{R}^p$ into regions $R_k = \{\mathbf{x} : f(\mathbf{x}) = k\}$; the decision boundary is where regions meet.
- For two classes, the Bayes decision boundary is $\{\mathbf{x} : p_1(\mathbf{x}) = 0.5\}$.
- A linear boundary is a hyperplane, $\boldsymbol{\beta}^\top \tilde{\mathbf{x}} = 0$; a nonlinear boundary is curved.



Logistic Regression

Modeling a Binary Response

- Linear regression models the mean of a quantitative response given the features: $\mathbb{E}[y \mid \mathbf{x}] = \boldsymbol{\beta}^\top \tilde{\mathbf{x}}$, with random errors around this mean.
- A binary response $y \in \{0, 1\}$ given $\mathbf{x}$ has a Bernoulli distribution: $P(y = 1 \mid \mathbf{x}) = p(\mathbf{x})$ and $P(y = 0 \mid \mathbf{x}) = 1 - p(\mathbf{x})$.
- The mean of a Bernoulli variable is its success probability: $\mathbb{E}[y \mid \mathbf{x}] = 1 \cdot p(\mathbf{x}) + 0 \cdot (1 - p(\mathbf{x})) = p(\mathbf{x})$.
- So, as in linear regression, we model how the mean of y depends on $\mathbf{x}$. Here the mean is a probability, so it must stay in $[0, 1]$.

The Logistic Function

- Keep a linear score $z = \boldsymbol{\beta}^\top \tilde{\mathbf{x}} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$, which can take any real value, and pass it through a function that maps $\mathbb{R}$ to $(0, 1)$: the logistic (sigmoid) function $\sigma(z) = 1/(1 + e^{-z})$.
- The logistic regression model:

$$p(\mathbf{x}) = \sigma(\boldsymbol{\beta}^\top \tilde{\mathbf{x}}) = \frac{1}{1 + e^{-\boldsymbol{\beta}^\top \tilde{\mathbf{x}}}}, \qquad \boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^\top.$$

- Properties: $\sigma(z) \in (0, 1)$, $\sigma(0) = 0.5$, σ is increasing, and $\sigma(-z) = 1 - \sigma(z)$.
- Fitted probabilities $\hat{p}(\mathbf{x})$ are therefore always valid, and the fitted curve is S-shaped rather than a straight line.

Odds

- The odds of an event compare the probability that it happens with the probability that it does not: $\frac{p(\mathbf{x})}{1-p(\mathbf{x})} \in (0, \infty)$.
- Examples: $p = 0.8$ gives odds of 4 (four to one in favour of $y = 1$), $p = 0.5$ gives odds of 1 (even odds), $p = 0.2$ gives odds of 0.25.
- Odds above 1 mean $y = 1$ is more likely than $y = 0$; odds below 1 mean it is less likely.

Log-Odds

- The log-odds, or logit, $\log \frac{p(\mathbf{x})}{1-p(\mathbf{x})} \in (-\infty, \infty)$, removes the lower bound of the odds.
- It is symmetric around $p = 0.5$, where the log-odds equal 0.
- The logit is the inverse of the logistic function.

probability p	0.1	0.2	0.5	0.8	0.9
odds	0.11	0.25	1	4	9
log-odds	-2.20	-1.39	0	1.39	2.20

Logistic Regression Is Linear in the Log-Odds

- With $z = \boldsymbol{\beta}^\top \tilde{\mathbf{x}}$ and $p(\mathbf{x}) = 1/(1+e^{-z})$, we have $1 - p(\mathbf{x}) = e^{-z}/(1+e^{-z})$, so

$$\frac{p(\mathbf{x})}{1-p(\mathbf{x})} = \frac{1}{e^{-z}} = e^{z}.$$

- Taking logs, the log-odds is linear in $\mathbf{x}$:

$$\log \frac{p(\mathbf{x})}{1-p(\mathbf{x})} = \boldsymbol{\beta}^\top \tilde{\mathbf{x}}.$$

- Logistic regression is therefore a linear regression for the log-odds: the logit connects the mean of y (a probability) to the linear score, the same idea as regression on a different scale.

Interpreting the Coefficients

- Increasing x_j by one unit with the other features fixed adds β_j to the log-odds, i.e. multiplies the odds by e^{β_j} (an odds ratio).
- If $\beta_j > 0$ the probability of $y = 1$ increases with x_j; if $\beta_j < 0$ it decreases; if $\beta_j = 0$ there is no association.
- The change in $p(\mathbf{x})$ itself is not constant: doubling the odds moves p from 0.10 to 0.18, from 0.50 to 0.67, and from 0.90 to 0.95.
- Coefficients are therefore interpreted on the log-odds or odds scale, not as changes in probability.

Likelihood Function

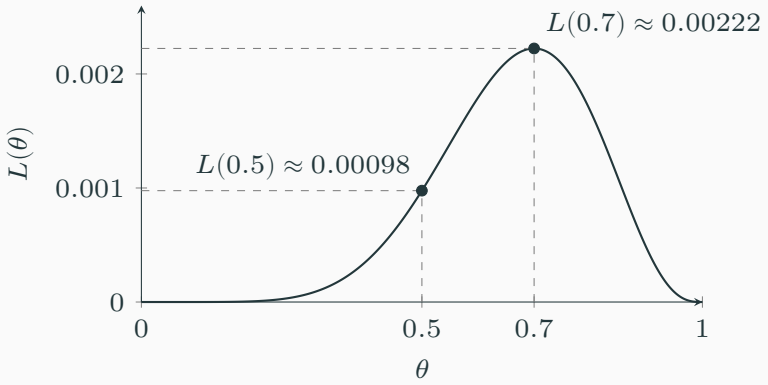
Estimation: The Likelihood Idea

- Least squares is built for a quantitative response with additive errors; for a Bernoulli response we use a different principle called maximum likelihood estimation.
- Idea: choose the parameter values under which the observed data would have been most probable.
- The likelihood is the probability of the observed data, viewed as a function of the parameters with the data held fixed.
- The maximum likelihood estimate is the parameter value that maximizes the likelihood.

Likelihood: A Coin-Flip Example

- An unfair coin lands heads with unknown probability θ. We flip it 10 times and observe a particular sequence with 7 heads and 3 tails.
- The flips are independent, so the probability of this sequence is the likelihood $L(\theta) = \theta^7(1-\theta)^3$.
- Comparing values: $L(0.5) \approx 0.00098$ and $L(0.7) \approx 0.00222$. The data are more than twice as probable under $\theta = 0.7$.
- The likelihood is maximized at $\hat{\theta} = 7/10$, the observed proportion of heads: maximum likelihood agrees with intuition.

Likelihood: The Coin-Flip Likelihood Curve



- The curve peaks at $\hat{\theta} = 0.7$, the observed proportion of heads.
- $L(0.7)/L(0.5) \approx 2.3$: the data are more than twice as probable under $\theta = 0.7$ than under a fair coin.

Bernoulli Likelihood of One Observation

- Given $\mathbf{x}^{(i)}$, the label is Bernoulli, $y^{(i)} \mid \mathbf{x}^{(i)} \sim \text{Bernoulli}(p^{(i)})$. The probability of the observed label has a single formula:

$$P(y^{(i)} \mid \mathbf{x}^{(i)}) = (p^{(i)})^{y^{(i)}} (1 - p^{(i)})^{1 - y^{(i)}}.$$

- It equals $p^{(i)}$ when $y^{(i)} = 1$ and $1 - p^{(i)}$ when $y^{(i)} = 0$.
- The regression model enters through the success probability: $p^{(i)} = p(\mathbf{x}^{(i)}) = \sigma(\boldsymbol{\beta}^\top \tilde{\mathbf{x}}^{(i)})$.
- So the probability the model assigns to each observed label depends on the parameter vector $\boldsymbol{\beta}$.

Likelihood of the Sample

- The observations are independent, so the probability of all n labels is the product of the individual probabilities:

$$\begin{aligned} L(\boldsymbol{\beta}) &= \prod_{i=1}^{n} \left(p^{(i)}\right)^{y^{(i)}} \left(1 - p^{(i)}\right)^{1-y^{(i)}} \\ &= \prod_{i:y^{(i)}=1} p^{(i)} \prod_{i:y^{(i)}=0} (1 - p^{(i)}). \end{aligned}$$

- L is large when $p^{(i)}$ is close to 1 for observations with $y^{(i)} = 1$ and close to 0 for observations with $y^{(i)} = 0$, i.e. when the coefficients fit the observed labels well.
- The maximum likelihood estimates are $\hat{\boldsymbol{\beta}} = \arg\max_{\boldsymbol{\beta}} L(\boldsymbol{\beta})$.

Why the Log-Likelihood?

- A product of n probabilities, each below 1, is extremely small and numerically unstable: $0.9^{1000} \approx 10^{-46}$.
- The logarithm is strictly increasing, so L and $\log L$ are maximized by the same parameter values.
- The logarithm turns the product into a sum, with one simple term per observation:

$$\ell(\boldsymbol{\beta}) = \log L = \sum_{i=1}^{n} \Big[y^{(i)} \log p^{(i)} + (1 - y^{(i)}) \log \left(1 - p^{(i)}\right) \Big].$$

- Sums are far easier to differentiate than products.

Maximizing the Log-Likelihood

- L is the probability the model assigns to the data we actually observed: a larger value means the parameters explain the data better. It is not the probability that the parameters are correct.
- First we have to derive the gradient of the log-likelihood function, which is called the Score function.
- Setting it to zero gives the score equations.
- $\mathbf{p}$ is nonlinear in $\boldsymbol{\beta}$, so there is no closed-form solution.
- ℓ is concave, so any solution is a global maximum. It is found iteratively by the Newton–Raphson method.

Score Function: Derivation

- With $z^{(i)} = \boldsymbol{\beta}^\top \tilde{\mathbf{x}}^{(i)}$ and $p^{(i)} = \sigma(z^{(i)})$, the chain rule gives, for $j = 0, \dots, p$,

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^{n} \frac{\partial \ell^{(i)}}{\partial p^{(i)}} \cdot \frac{\partial p^{(i)}}{\partial z^{(i)}} \cdot \frac{\partial z^{(i)}}{\partial \beta_j}.$$

- Expanding each term:

$$\begin{aligned}
\frac{\partial \ell^{(i)}}{\partial p^{(i)}} &= \frac{y^{(i)}}{p^{(i)}} - \frac{1 - y^{(i)}}{1 - p^{(i)}} = \frac{y^{(i)} - p^{(i)}}{p^{(i)}\big(1 - p^{(i)}\big)}, \\
\frac{\partial p^{(i)}}{\partial z^{(i)}} &= \frac{e^{-z^{(i)}}}{\big(1 + e^{-z^{(i)}}\big)^2} = \frac{1}{1 + e^{-z^{(i)}}} \cdot \frac{e^{-z^{(i)}}}{1 + e^{-z^{(i)}}} = p^{(i)}\big(1 - p^{(i)}\big), \\
\frac{\partial z^{(i)}}{\partial \beta_j} &= \tilde{x}_j^{(i)}, \qquad \tilde{x}_0^{(i)} = 1.
\end{aligned}$$

- The factor $p^{(i)}(1 - p^{(i)})$ cancels; stacking over $j = 0, \dots, p$ gives the score vector:

$$\frac{\partial \ell}{\partial \beta_j} = \sum_{i=1}^{n} \big(y^{(i)} - p^{(i)}\big)\tilde{x}_j^{(i)} \quad \Longrightarrow \quad \nabla \ell(\boldsymbol{\beta}) = \tilde{\mathbf{X}}^\top (\mathbf{y} - \mathbf{p}).$$

Hessian and Concavity

- The score equations have no closed-form solution, so we need the second derivatives of ℓ.
- Differentiating the score once more gives the entries of the Hessian:

$$\frac{\partial^2 \ell}{\partial \beta_j \, \partial \beta_k} = -\sum_{i=1}^{n} p^{(i)}(1 - p^{(i)}) \, \tilde{x}_j^{(i)} \tilde{x}_k^{(i)}.$$

- In matrix form, $\mathbf{H}(\boldsymbol{\beta}) = -\tilde{\mathbf{X}}^\top \mathbf{W} \tilde{\mathbf{X}}$, where $\mathbf{W} = \operatorname{diag}\left(p^{(i)}(1 - p^{(i)})\right)$ is an $n \times n$ diagonal matrix of positive weights.
- With positive weights and full column rank $\tilde{\mathbf{X}}$, $\mathbf{H}$ is negative definite, so ℓ is concave.
- So the score equations have at most one solution, and it is the global maximum.

Newton–Raphson

- Idea: at the current $\boldsymbol{\beta}$, replace ℓ by a quadratic that matches its gradient and Hessian, then jump to the maximum of that quadratic.
- This gives the update

$$\boldsymbol{\beta}^{\text{new}} = \boldsymbol{\beta} - \mathbf{H}^{-1}\nabla\ell = \boldsymbol{\beta} + (\tilde{\mathbf{X}}^\top \mathbf{W}\tilde{\mathbf{X}})^{-1}\tilde{\mathbf{X}}^\top(\mathbf{y} - \mathbf{p}).$$

- Algorithm: start from $\boldsymbol{\beta} = \mathbf{0}$, compute $\mathbf{p}$ and $\mathbf{W}$, apply the update, and repeat until $\boldsymbol{\beta}$ stops changing.
- $\mathbf{p}$ and $\mathbf{W}$ depend on $\boldsymbol{\beta}$, so they are recomputed at every step.

Iteratively Reweighted Least Squares

- The Newton update can be written as a weighted least-squares fit:

$$\boldsymbol{\beta}^{\text{new}} = (\tilde{\mathbf{X}}^\top \mathbf{W} \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{W} \mathbf{z}, \qquad \mathbf{z} = \tilde{\mathbf{X}}\boldsymbol{\beta} + \mathbf{W}^{-1}(\mathbf{y} - \mathbf{p}).$$

- Compare with ordinary least squares, $(\tilde{\mathbf{X}}^\top \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^\top \mathbf{y}$: the same formula, with weights $\mathbf{W}$ and working response $\mathbf{z}$ in place of $\mathbf{y}$.
- The weights $p^{(i)}(1 - p^{(i)})$ are largest at $p^{(i)} = 0.5$ and small near 0 or 1.
- $\mathbf{W}$ and $\mathbf{z}$ change at every iteration, hence “iteratively reweighted”.

Making Predictions

- For a new observation $\mathbf{x}^*$, the fitted probability is $\hat{p}(\mathbf{x}^*) = \sigma(\hat{\boldsymbol{\beta}}^\top \tilde{\mathbf{x}}^*)$.
- Classify with a threshold τ: predict $\hat{y}^* = 1$ if $\hat{p}(\mathbf{x}^*) > \tau$, else $\hat{y}^* = 0$.
- The default $\tau = 0.5$ is the Bayes rule with the estimated probability in place of the true one.
- A lower τ labels more observations as class 1 (for example, flags more customers as likely defaulters); the trade-off between the two kinds of error is treated in a later lecture.

Worked Example: Default Data

- `Default`: $10{,}000$ credit card customers, response `default` (333 defaults), predictors `balance`, `income`, `student`.

	Estimate	Std. error	z	p-value
Intercept	-10.6513	0.3612	-29.49	$< 2 \times 10^{-16}$
`balance`	0.00550	0.00022	24.95	$< 2 \times 10^{-16}$

Interpreting the Default Fit

- $e^{100\hat{\beta}_1} \approx 1.73$: each additional \$100 of balance multiplies the odds of default by about 1.73 (95% CI $[1.66, 1.81]$).
- The coefficient of `balance` is positive and highly significant ($z = 24.95$): higher balances go with a higher probability of default.
- The intercept is the log-odds of default at zero balance: $\hat{p}(\text{balance} = 0) \approx 0.00002$.
- Predictions: $\hat{p}(1000) = \sigma(-10.6513 + 0.00550 \times 1000) \approx 0.006$ and $\hat{p}(2000) \approx 0.586$.

Multiple Logistic Regression

- Additional predictors enter the log-odds linearly:

$$\log \frac{p(\mathbf{x})}{1-p(\mathbf{x})} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = \boldsymbol{\beta}^\top \tilde{\mathbf{x}}.$$

- Each β_j is the effect of x_j on the log-odds holding the other predictors fixed, and e^{β_j} is the corresponding odds ratio.
- A categorical predictor enters through dummy variables, for example $x = 1$ for a student and $x = 0$ otherwise.

Confounding: Student Status in the Default Data

- With student as the only predictor, its coefficient is $+0.405$ ($p < 0.001$): students default more often.
- With balance and income included, it is -0.647 ($p = 0.006$): a student defaults less often than a non-student with the same balance and income.
- Students carry higher balances on average (about \$988 vs. \$772), and higher balances raise default risk.
- As in multiple linear regression, the marginal and the conditional effect can differ in sign.

Classification with More than Two Classes

Multinomial Regression

- For $K > 2$ classes, model every class probability with the softmax function:

$$P(y = k \mid \mathbf{x}) = \frac{\exp(\boldsymbol{\beta}_k^\top \tilde{\mathbf{x}})}{\sum_{l=1}^{K} \exp(\boldsymbol{\beta}_l^\top \tilde{\mathbf{x}})}, \qquad k = 1, \dots, K.$$

- Each class has its own coefficient vector $\boldsymbol{\beta}_k = (\beta_{k0}, \beta_{k1}, \dots, \beta_{kp})^\top$, including an intercept. The probabilities are positive and sum to one.
- Boundaries between any pair of classes remain linear in $\mathbf{x}$, since $P(y = k \mid \mathbf{x}) = P(y = l \mid \mathbf{x})$ reduces to $\boldsymbol{\beta}_k^\top \tilde{\mathbf{x}} = \boldsymbol{\beta}_l^\top \tilde{\mathbf{x}}$.

Reference Class and Interpretation

- We need a reference class: fix one class r and set $\boldsymbol{\beta}_r = \mathbf{0}$. The other $K-1$ classes keep their own coefficient vectors.
- Relative to the reference class, for every class $k \neq r$,

$$\log \frac{P(y=k \mid \mathbf{x})}{P(y=r \mid \mathbf{x})} = \boldsymbol{\beta}_k^\top \tilde{\mathbf{x}}.$$

- So β_{kj} is the change in the log-odds of class k versus class r per unit increase in x_j with the other features fixed, and $e^{\beta_{kj}}$ multiplies the odds of class k relative to class r, which is why $e^{\beta_{kj}}$ is also called the odds ratio corresponding to feature j.
- The coefficients depend on the choice of reference class; the fitted probabilities do not.
- For $K = 2$ with class 2 as the reference, $P(y=1 \mid \mathbf{x}) = \sigma(\boldsymbol{\beta}_1^\top \tilde{\mathbf{x}})$: ordinary logistic regression.

k-Nearest Neighbors

k-Nearest Neighbors Classifier

- k-NN is nonparametric: there is no model to fit, and predictions use the training data directly.
- For a new point $\mathbf{x}^*$ and a chosen integer k, let $\mathcal{N}^*$ be the set of indices of the k training points closest to $\mathbf{x}^*$.
- Estimate the class probabilities by the class proportions in the neighbourhood:

$$\hat{p}_j(\mathbf{x}^*) = \frac{1}{k} \sum_{i \in \mathcal{N}^*} \mathbb{1}\{y^{(i)} = j\}.$$

k-NN: Majority Vote

- Classify by majority vote, $\hat{y}^* = \arg\max_j \hat{p}_j(\mathbf{x}^*)$: the Bayes rule applied to the estimated probabilities.
- Ties are broken at random.
- Example: with $k = 3$ and two of the three nearest neighbours in class 1, $\hat{p}_1(\mathbf{x}^*) = 2/3$ and $\hat{y}^* = 1$.

Distance Metrics

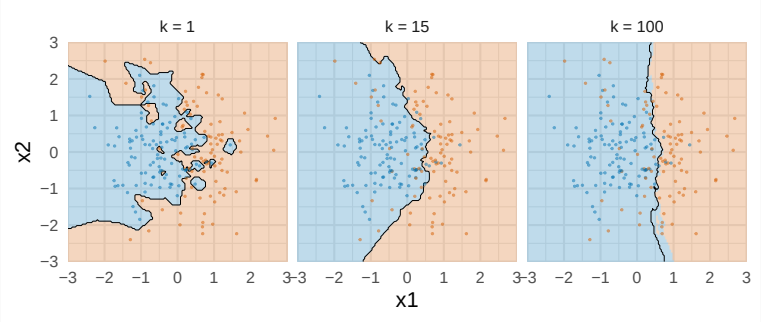
- “Closest” requires a distance, and different distances can give different neighbours.
- The default is Euclidean, $d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_{j=1}^{p} (a_j - b_j)^2}$.
- Alternatives: Manhattan, $\sum_{j=1}^{p} |a_j - b_j|$, and the Minkowski family $\left(\sum_{j=1}^{p} |a_j - b_j|^q \right)^{1/q}$, which contains both ($q = 1$ and $q = 2$).

Feature Scaling

- Features with a large spread dominate the distance: in `Default`, `income` has a standard deviation of about $13,300 and `balance` about $480, so raw distances are driven almost entirely by `income`, the weaker predictor of default.
- Standardize before computing distances, $z_j^{(i)} = (x_j^{(i)} - \bar{x}_j)/s_j$, where $\bar{x}_j$ and s_j are the sample mean and standard deviation of feature j.
- Every standardized feature has mean 0 and standard deviation 1, so all features contribute on a comparable scale.
- Preprocessing is treated in detail in a later lecture.

The Effect of k

- Small k ($k = 1$): jagged boundary following individual points; training error is 0, which says nothing about new data.
- Large k: smooth, rigid boundary that can miss genuine curvature; $k = n$ predicts the majority class everywhere.
- k is a tuning parameter; choosing it needs held-out data (a later lecture).



k-NN: Strengths

- No fitting step: predictions use the stored training data directly.
- No assumption about the shape of the decision boundary.
- A single tuning parameter k, and easy to explain.

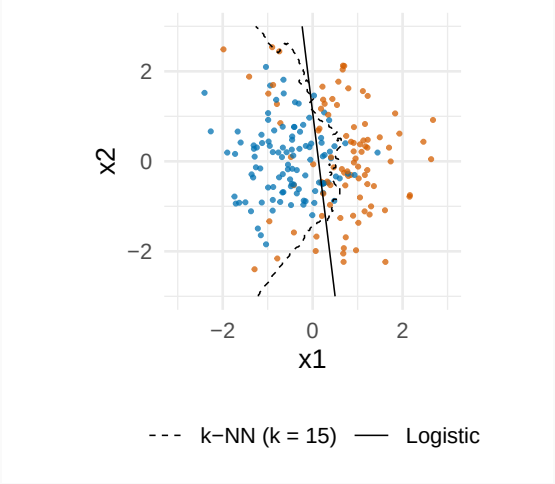
k-NN: Weaknesses

- Cost: every prediction computes distances to all n training points, $O(np)$ per query, and the whole training set must be stored.
- Curse of dimensionality: as p grows the data become sparse and the "nearest" neighbours are no longer near; irrelevant features add noise to every distance.
- No coefficients: k-NN gives predictions but no summary of how each feature affects the outcome, unlike e^{β_j} in logistic regression.
- Sensitive to feature scaling.

Comparison and Summary

Logistic Regression vs. k-NN on the Same Data

- Same simulated data (curved true boundary); boundaries drawn at $\hat{p} = 0.5$.
- Logistic regression: a straight line, misses the curvature.
- k-NN ($k = 15$): follows the curve, locally jagged.
- Which is better depends on the true boundary and the sample size; judging it needs held-out data.



Logistic Regression vs. k-NN

	Logistic regression	k-NN
Type	Parametric: $\boldsymbol{\beta}$	Nonparametric: stores the training data
Decision boundary	Linear, unless features are added	Flexible, controlled by k
Fitting	Iterative maximum likelihood	None
Cost per prediction	$O(p)$	$O(np)$
Interpretation	Odds ratios e^{β_j}, z-tests, confidence intervals	No coefficients
Main weakness	Misses nonlinear structure unless features are engineered	Feature scaling, irrelevant features, high dimensions

Summary: Classification and Logistic Regression

- Classification predicts a category by estimating the class probabilities $p_k(\mathbf{x})$. The Bayes classifier (most probable class) is the unattainable benchmark, and the Bayes error rate is irreducible.
- Logistic regression models the Bernoulli success probability through the log-odds, $\log \frac{p(\mathbf{x})}{1-p(\mathbf{x})} = \boldsymbol{\beta}^\top \tilde{\mathbf{x}}$; coefficients are read as odds ratios e^{β_j}.
- The coefficients maximize the likelihood, equivalently the log-likelihood or minus the cross-entropy, found iteratively; the decision boundary is linear.
- It extends to $K > 2$ classes through the multinomial model with a reference class, and to nonlinear boundaries through added features.

Summary: k-NN and Next Steps

- k-NN: majority vote among the k closest training points, a flexible boundary controlled by k; it needs feature scaling and degrades in high dimensions.
- Logistic regression is parametric, interpretable and linear; k-NN is nonparametric and flexible but has no coefficients.
- Still open: how to choose k and how to compare classifiers on data not used for fitting.

Questions?